Experiment 5 Siddhant Bajaj

Aim: Study of solidity program structure, basic constructs compilation and deployment environment

Lab Objectives

- To explore fundamental concepts of Blockchain technology.
- To understand the basics of Solidity programming.
- To learn how Smart Contracts are created and executed on the Ethereum Blockchain.

Lab Outcome (LO)

LO2: Design and develop Smart Contracts using Solidity.

Theory

1. Primitive Data Types, Variables, and Functions – pure and view

Solidity provides several primitive data types that form the foundation of Smart Contract development. The commonly used primitive data types are:

- uint / int – Used to store unsigned and signed integers of different sizes, such as uint256 and int128.
- bool – Used to represent logical values such as true or false.
- address – Used to store a 20-byte Ethereum account or Smart Contract address.
- bytes – Used to store binary data.
- string – Used to store textual data.

Types of Variables

Variables in Solidity can be classified into the following types:

- State Variables: Variables whose values are permanently stored on the blockchain.

- Local Variables: Temporary variables that exist only during the execution of a function.
- Global Variables: Special variables provided by Solidity, such as msg.sender, msg.value, and block.timestamp.

Functions

Functions are used to execute specific operations in a Smart Contract. Two important function types are:

- pure function: A pure function cannot read or modify the state of the blockchain. It operates only on the input parameters and internal calculations.
- view function: A view function can read state variables but cannot modify them.

Using pure and view appropriately helps ensure that functions behave as intended and can avoid unnecessary state-changing operations.

2. Inputs and Outputs to Functions

Solidity functions can accept input parameters and return one or more output values.

Inputs

Input parameters allow users or other Smart Contracts to pass data to a function. For example, a function may accept an integer representing an amount or a string representing a name.

Outputs

Output parameters allow a function to return the result of an operation. Solidity also supports named return variables, which improve code readability and make the function easier to understand.

3. Visibility, Modifiers, and Constructors

Function Visibility

Function visibility determines who can access a function.

- public – The function can be called both internally and externally.
- private – The function can be accessed only within the contract in which it is defined.

- internal – The function can be accessed within the contract and its derived contracts.
- external – The function is intended to be called from outside the contract.

Modifiers

Modifiers are reusable blocks of code that can change or restrict the behavior of functions. They are commonly used for access control.

For example, an onlyOwner modifier can restrict a function so that only the contract owner can execute it.

Constructors

A constructor is a special function that is executed only once, at the time of Smart Contract deployment.

Constructors are generally used to initialize important state variables, such as assigning the address of the account that deployed the contract as its owner.

4. Control Flow: if-else and Loops

Solidity provides control-flow statements similar to those found in other programming languages.

if-else

The if-else statement is used to make decisions based on conditions.

Loops

Solidity supports loops such as:

- for
- while
- do-while

Loops are useful when a particular operation needs to be repeated, such as iterating through an array of users.

However, loops should be used carefully in Smart Contracts because every operation consumes gas. A loop with a large number of iterations can make a transaction expensive or may cause it to exceed the block gas limit.

5. Data Structures: Arrays, Mappings, Structs, and Enums

Solidity provides several data structures for organizing and storing information.

Arrays

Arrays are ordered collections of elements. They can be:

- Fixed-size arrays
- Dynamic arrays

Mappings

Mappings store data in key-value pairs and provide efficient access to values using their keys.

Structs

Structures (struct) allow multiple related variables to be grouped into a single custom data type.

Enums

Enums are used to define a set of predefined constant values. They make code easier to understand and improve readability.

6. Data Locations

Solidity provides three primary data locations:

storage

- Stores data permanently on the blockchain.
- State variables are generally stored in storage.
- Changes to storage consume gas because the blockchain state is modified.

memory

- Used for temporary data during function execution.
- Data stored in memory exists only while the function is executing.
- It is commonly used for temporary copies of arrays, structs, and other reference types.

calldata

- Used for function parameters of external functions.
- It is read-only and cannot be modified.
- It is generally more gas-efficient than copying data into memory.

Understanding data locations is important because they directly affect the gas consumption, performance, and behavior of Smart Contracts.

7. Transactions: Ether and Wei, Gas and Gas Price, and Sending Transactions

Ether and Wei

Ether (ETH) is the native cryptocurrency of the Ethereum network.

The smallest unit of Ether is called Wei.

[
1\ Ether = 10^{18}\ Wei
]

Using Wei allows Ethereum to represent very small values accurately.

Gas and Gas Price

Gas represents the computational work required to execute a transaction or Smart Contract operation.

The gas price determines how much ETH is paid for each unit of gas.

The total transaction fee can be represented as:

[

$$Transaction\ Fee = Gas\ Used \times Gas\ Price$$

]

Efficient Smart Contract design is important because unnecessary computations and storage operations can increase gas consumption.

Sending Transactions

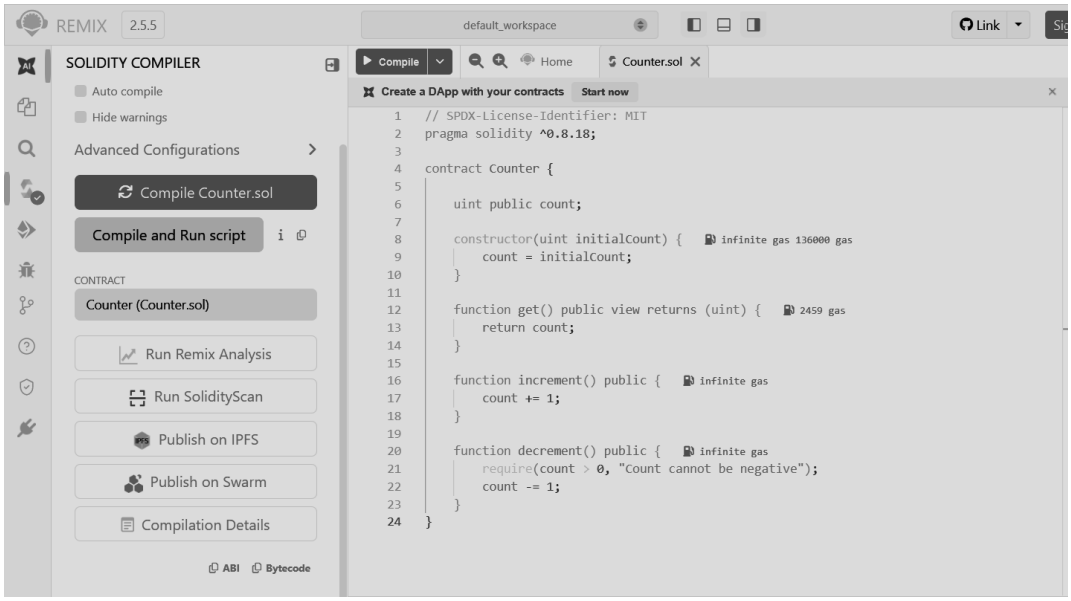
Transactions are used to:

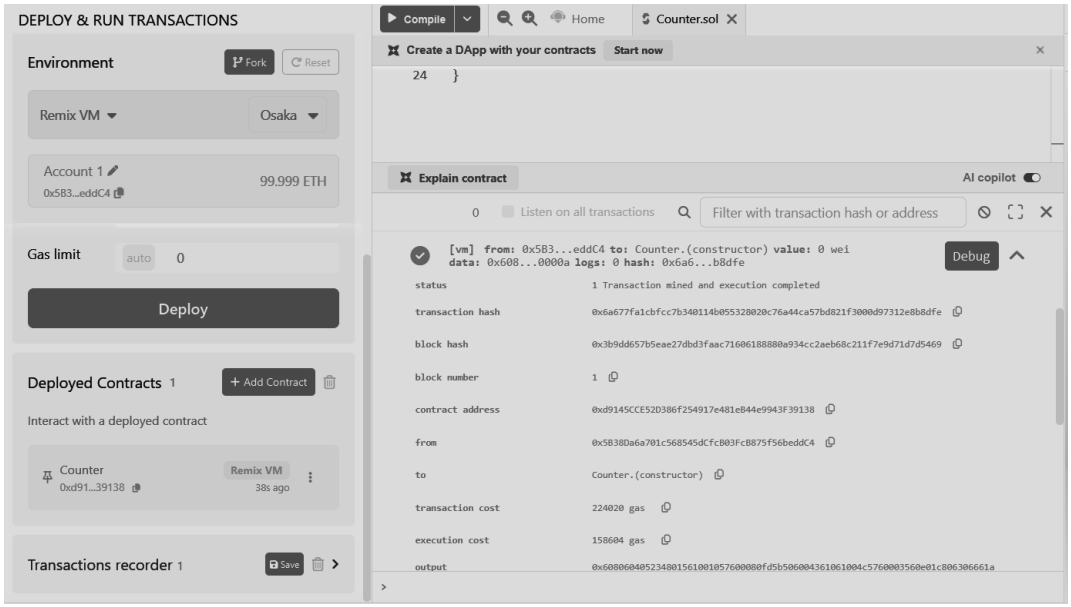
- Transfer Ether between accounts.
- Interact with Smart Contracts.
- Modify blockchain state.
- Execute payable functions.

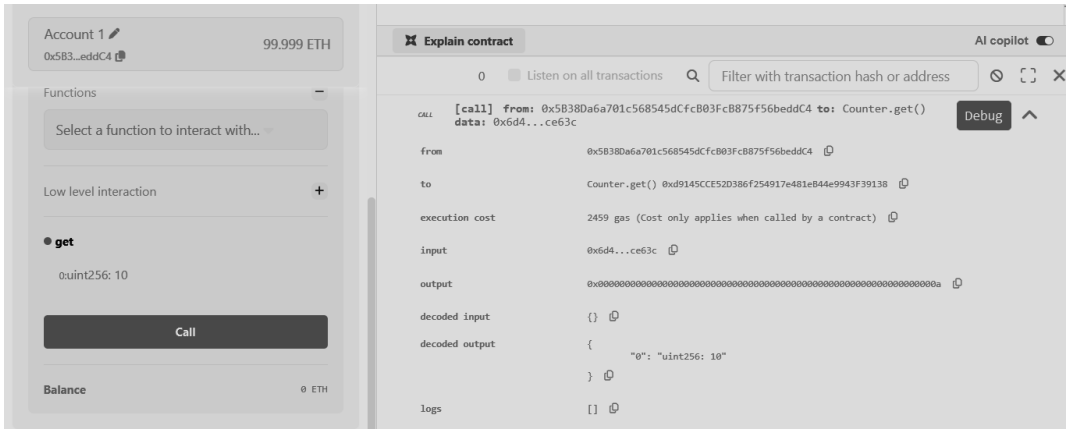
Solidity provides several mechanisms for sending Ether, including:

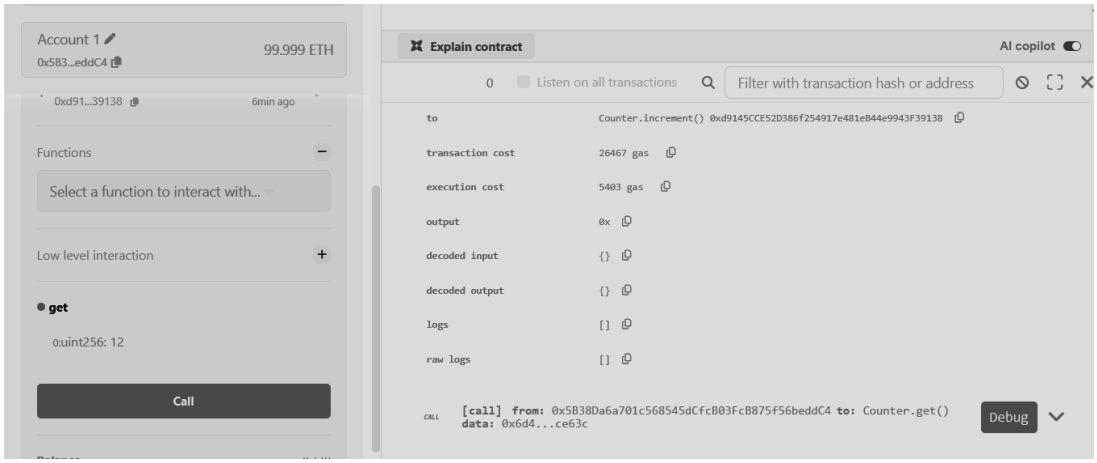
- transfer()
- send()
- call()

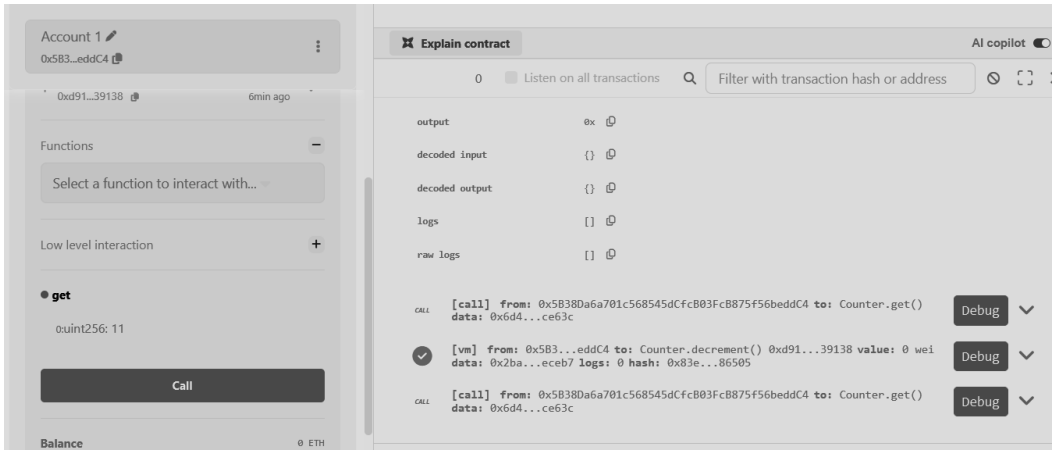
The call() method provides greater flexibility and is commonly used for low-level Ether transfers. Ether transfers and state-changing contract interactions require gas.

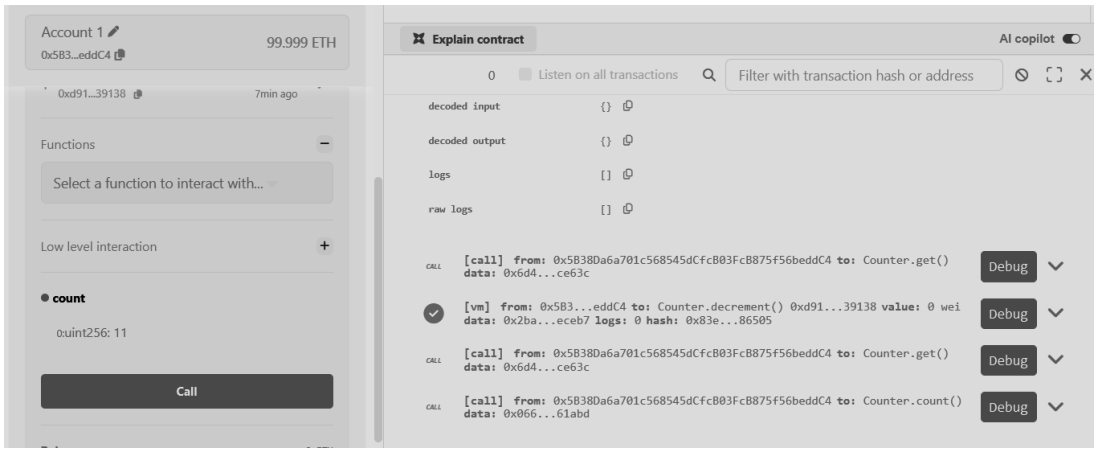



DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Gas limit
auto
0
Deploy
Deployed Contracts 1
+ Add Contract
Interact with a deployed contract
Counter
0xd91...39138
Remix VM
38s ago
Transactions recorder 1
Save
Compile
Home
Counter.sol
Create a DApp with your contracts
Start now
24 }
Explain contract
AI copilot
0
Listen on all transactions
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei
data: 0x608...0000a logs: 0 hash: 0x6a6...b8dfe
Debug
status
1 Transaction mined and execution completed
transaction hash
0x6a677fa1cbfcc7b340114b05532020c76a44ca57bd821f3000d97312e8b8dfe
block hash
0x3b9dd657b5eae27dbd3faac71606188880a934cc2aeb68c211f7e9d71d7d5469
block number
1
contract address
0xd9145CCE52D386f254917e481eB44e9943F39138
from
0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to
Counter.(constructor)
transaction cost
224020 gas
execution cost
158604 gas
output
0x6080604052348015610010576000080fd5b506004361061004c5760003560e01c806306661a


Account 1
0x5B3...eddC4
99.999 ETH
Functions
Select a function to interact with...
Low level interaction
get
0:uint256: 10
Call
Balance
0 ETH
Explain contract
AI copilot
0
Listen on all transactions
Filter with transaction hash or address
call [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get()
data: 0x6d4...ce63c
Debug
from
0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to
Counter.get() 0xd9145CCE52D386f254917e481eB44e9943F39138
execution cost
2459 gas (Cost only applies when called by a contract)
input
0x6d4...ce63c
output
0x000000000000000000000000000000000000000000000000000000000000000a
decoded input
{}
decoded output
{
"0": "uint256: 10"
}
logs
[]


Account 1
0x5B3...eddC4
99.999 ETH
0xd91...39138
6min ago
Functions
Select a function to interact with...
Low level interaction
get
0:uint256: 12
Call
Explain contract
AI copilot
0
Listen on all transactions
Filter with transaction hash or address
to
Counter.increment() 0xd9145CCE52D386f254917e481eB44e9943F39138
transaction cost
26467 gas
execution cost
5403 gas
output
0x
decoded input
{}
decoded output
{}
logs
[]
raw logs
[]
call [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get()
data: 0x6d4...ce63c
Debug





Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.